

PROJETO FROTA MISTA v2

Documentação Integrada de Engenharia

PRD · SPEC · Workflow · Diagramas de Circuito · Guia de Setup

Prof. Francenylson Luiz Dantas dos Santos

Projeto Aluno Maker Digital — CRE Recanto das Emas / DF

Versão: v6 — 30/05/2026 | Python Puro (sem ROS) | Raspberry Pi 5

Repositório: github.com/francenylson1/robo_slam_v2

1. PRD — Product Requirement Document

1.1 Visão Geral do Produto

Ecossistema unificado em Python para gerenciar e operar uma frota mista de 10 robôs garçons construídos sobre motores de hoverboard, controlados por drivers ZS-X11H V2. Projeto com forte viés de tecnologia assistiva e inclusão escolar.

 **REGRA DE SEGURANÇA Nº 0** — Clipping rígido a 15% da potência nominal. Leitura $\geq 20\%$ → Emergency Stop imediato e desligamento total.

1.2 Objetivos Estratégicos

- Inclusão e Protagonismo: alunos cadeirantes operam robôs via joystick com vídeo em tempo real.
- Segurança Ativa Rígida (Regra 0): clipping de 15% e Emergency Stop automático em todos os caminhos de comando.
- Interface Responsiva do Zero: 34" ultrawide → 15.6" frontal → 7" touchscreen → smartphones.
- Independência de Infraestrutura: funcionamento 100% offline, sem nuvem ou internet externa.
- Aproveitamento estratégico do legado v1: pinos GPIO, lógica direcional e PID calibrado migrados e preservados.

1.3 Escopo da Frota

Tipo	Qtd	Modo de Operação	Controle
Autônomo (SLAM)	3	Navegação por POIs via Aurora	slam_nav.py
Assistivo (Joystick)	7	Teleoperação em tempo real	Dongle USB 2.4GHz (<10ms)

1.4 Requisitos Funcionais

ID	Requisito	Prioridade
RF01	Boot por ID: config.py determina modo AUTONOMO ou JOYSTICK na inicialização	Must
RF02	Stream MJPEG via OpenCV transmitido pelo Flask para o navegador do operador	Must
RF03	Interface de voz: botões disparam .mp3/.wav no speaker ativo do robô	Should
RF04	Expressão facial: SVG dinâmico no display 7" sincronizado via WebSockets com áudios	Should
RF05	Telemetria de bateria: tensão real e % de carga exibidos no dashboard a cada 5s	Must
RF06	Gerenciador de POIs: marcar, nomear e salvar localizações em pois.json	Should

1.5 Decisão de Arquitetura — Aproveitamento do Legado v1

✓ **DECISÃO FORMAL:** Os scripts de controle motor do robo_slam v1 (commit d3727e0 / tag v1.0-estavel-base) foram analisados, validados e migrados para o v2. O núcleo de pinos GPIO, lógica direcional e ganhos PID NÃO serão reescritos — apenas encapsulados na nova arquitetura com a Regra 0 e modo MOCK por cima.

O que foi migrado	De onde	Onde está no v2	Status
Pinos GPIO dos motores	robot_motor_controller.py	config/settings.py	✓ Validado
Lógica direcional (HIGH/LOW)	gpio_test.py + motor_controller	core/motor_driver.py	✓ Validado
PIDController (Kp=0.26, Ki=0.23)	pid_controller.py	core/pid_controller.py	✓ Calibrado
Encoders Hall (polling 1000Hz)	robot_motor_controller.py	core/motor_driver.py	✓ Validado
JoystickController (pygame)	joystick_controller.py	core/joystick_reader.py	✓ Migrado
Detecção Pi vs PC (MOCK)	environment.py	config/settings.py	✓ Migrado

2. SPEC — Especificação Técnica de Arquitetura

2.1 Estrutura de Pastas do Repositório

Repositório: github.com/francenylson1/robo_slam_v2 | Branch principal: main | Legado
arquivado: github.com/francenylson1/robo_slam (tag: legacy-v1-referencia)

Pasta / Arquivo	Responsabilidade
main.py	Ponto de entrada — loop de controle 50Hz
config/settings.py	Toda configuração centralizada (pinos, PID, Regra 0, I2C, Flask)
core/motor_driver.py	⚠ NÚCLEO VALIDADO — pinos, Regra 0, PID, encoders Hall
core/pid_controller.py	PID com anti-windup — migrado do v1 sem alteração de lógica
core/joystick_reader.py	Dongle USB 2.4GHz via pygame — cinemática diferencial
sensors/battery_monitor.py	ADS1115 → tensão/% bateria (atualiza a cada 5s)
sensors/safety_bumper.py	RPLIDAR C1 → flag bloqueio_frontal @50Hz
sensors/heading_lock.py	BNO085 → Yaw em quatérniões → correção de linha reta
slam/poi_manager.py	Leitura/escrita pois.json sem restart
slam/slam_nav.py	Navegação autônoma Aurora SDK (Fase 4)
web/server.py	Flask + MJPEG + WebSocket telemetria
web/templates/dashboard.html	Interface responsiva (Grid/Flexbox)
docs/NUCLEO_MOTOR.md	Documentação formal dos pinos e lógica validada
PROMPT_INICIAL.md	Contexto completo para o Claude Code no Cursor
GUIA_SETUP.md	Passo a passo de instalação e configuração

2.2 Camadas de Comunicação

Linha	Caminho	Latência	Independência
Comando (Movimento)	Joystick → Dongle USB 2.4GHz → Pi 5 GPIO	< 10ms	Independente do Wi-Fi
Percepção (Vídeo/UI)	Pi 5 → Flask MJPEG → Wi-Fi 5GHz → Navegador	< 200ms	Só exibição — nunca controle
Dados SLAM	Aurora USB → Python SDK → Mapa local 2D	Local	100% offline
Telemetria	Sensores → main.py → WebSocket → Dashboard	5s	Rede local apenas

2.3 Núcleo Motor — Pinos GPIO Validados

⚠ NÃO ALTERAR — valores validados fisicamente no robô real (robo_slam v1). Qualquer mudança requer teste físico obrigatório antes de ser aceita no repositório.

Motor	Sinal	Pino BCM	Direção FRENTE	Direção TRÁS
Esquerdo	DIR_E	5	HIGH	LOW
Esquerdo	BREAK_E	6	LOW (livre)	HIGH (freado)
Esquerdo	PWM_E	18	0–15% duty cycle	20Hz
Esquerdo	HALL_E	16	Entrada PUD_DOWN	polling 1000Hz
Direito	DIR_D	23	LOW ← OPOSTO	HIGH ← OPOSTO
Direito	BREAK_D	24	LOW (livre)	HIGH (freado)
Direito	PWM_D	12	0–15% duty cycle	20Hz
Direito	HALL_D	17	Entrada PUD_DOWN	polling 1000Hz

2.4 Subsistema Elétrico — PDU

- Bateria: 42V máx / 36V nominal / 4400mAh.
- SZBK07: 42V → 5.1–5.2V / 20A. Alimenta Pi 5, displays e speaker. USB da Pi: somente dados.
- Step-down 2: 42V → 12V, dedicado exclusivamente ao Slamtec Aurora.
- Optoacopladores (PC817): todas as linhas PWM/DIR/BREAK → ZS-X11H V2.
- GND lógico (Pi 5 / 3.3V) e GND de potência (SZBK07 / 5.1V) nunca conectados diretamente.
- I2C compartilhado (pinos 3 e 5): BNO085 → 0x4A | ADS1115 → 0x48. Cabo máximo 30cm.
- Divisor resistivo: $R1=100k\Omega + R2=6.8k\Omega \rightarrow V_{out_max}=2.67V$ para $V_{bat}=42V$.

2.5 Ambiente de Desenvolvimento

Componente	Ferramenta	Detalhe
IDE	Cursor	Remote SSH conectado direto na Raspberry Pi 5
IA Integrada	Claude Code (Sonnet 4.6)	Agente de desenvolvimento dentro do Cursor
Controle de Versão	GitHub — robo_slam_v2	Commits somente em milestones estáveis
Contexto para IA	PROMPT_INICIAL.md	Cole na primeira mensagem ao Claude Code
OS da Pi	RaspiOS 64-bit	Python 3.11+
Rede	Wi-Fi 5GHz local	SSID oculto, sem WAN — TP-Link Archer AX73

3. Workflow — Cronograma por Fases

Cada fase é uma entrega fechada com Gate de Conclusão. Nenhuma fase se inicia sem que a anterior esteja 100% validada e em pleno funcionamento. O Claude Code no Cursor é o agente de desenvolvimento a partir da Fase 1.

Fase 0 — Fundação e Setup (1–2 dias) ✓

Esta fase está CONCLUÍDA. A estrutura do repositório robo_slam_v2 foi criada com todos os módulos base, configuração centralizada e documentação dos pinos validados do v1.

- ✓ Análise e triagem do repositório legado (robo_slam v1)
- ✓ Decisão formal de migração do núcleo motor (pinos, PID, encoders)
- ✓ Criação do repositório robo_slam_v2 com estrutura modular
- ✓ Migração dos scripts validados: motor_driver, pid_controller, joystick_reader
- ✓ Documentação formal dos pinos em docs/NUCLEO_MOTOR.md
- ✓ PROMPT_INICIAL.md e GUIA_SETUP.md para onboarding do Claude Code
- ⌚ Criar repo no GitHub, clonar na Pi, configurar Cursor + Remote SSH

Gate de Conclusão — critérios obrigatórios antes de avançar para a próxima fase:

- Repositório robo_slam_v2 criado no GitHub com primeiro commit
- Repositório legado robo_slam v1 arquivado com tag legacy-v1-referencia
- SSH funcionando: Cursor conectado à Pi via Remote SSH
- Tensões do SZBK07 medidas: 5.1–5.2V e 12V confirmadas com multímetro
- i2cdetect lista 0x48 (ADS1115) e 0x4A (BNO085) sem conflito
- python3 main.py --mock roda sem erros e dashboard abre no navegador

Fase 1 — Percepção e Telemetria (5–7 dias)

Desenvolvida no Cursor + Claude Code. Todos os sensores em modo MOCK primeiro, depois real.

- battery_monitor.py: leitura ADS1115 → tensão real → % de carga com curva de descarga.
- safety_bumper.py: RPLIDAR C1 → zona frontal <50cm → flag bloqueio_frontal a 50Hz.
- heading_lock.py: BNO085 → Yaw em quatérniões → cálculo de desvio angular.
- motor_driver.py modo MOCK: registra comandos em log — nenhuma saída física em GPIO.
- Loop 50Hz (20ms) em main.py: threading e queue validados com medição de jitter.

Gate de Conclusão — critérios obrigatórios antes de avançar para a próxima fase:

- Leitura de tensão com precisão $\pm 0.5V$ comparada ao multímetro
- LIDAR bloqueia flag bloqueio_frontal com objeto posicionado a 45cm
- BNO085 retorna Yaw estável por 60s sem drift visível
- Loop 50Hz: jitter máximo de 5ms medido com time.perf_counter()
- python3 main.py --log DEBUG mostra todos os subsistemas rodando

Fase 2 — Interface Web Responsiva e Expressão Facial (5–8 dias)

- web/server.py: Flask com Flexbox/Grid validado em 34", 15.6", 7" e smartphones.
- Stream MJPEG: OpenCV testado com 3 browsers simultâneos sem freeze por 30s.
- face_engine.py: SVG animado sincronizado via WebSockets com áudios locais no P2.
- Interface inclusiva: botões de frase rápida → reprodução imediata no speaker 6W.
- poi_manager.py: marcar POI → salvar pois.json → recarregar sem restart do servidor.
- Telemetria em tempo real: tensão + % bateria atualizando no dashboard via WebSocket.

Gate de Conclusão — critérios obrigatórios antes de avançar para a próxima fase:

- Dashboard renderiza corretamente nos 4 formatos de tela (34", 15.6", 7", smartphone)
- Stream MJPEG sem freeze por 30s consecutivos em Wi-Fi 5GHz local
- Expressão facial muda em sincronia com o áudio reproduzido
- POI gravado e recuperado corretamente após restart do servidor
- Telemetria atualiza no dashboard a cada 5s sem travar a interface

Fase 3 — Integração de Potência no Chassi Real (7–10 dias)

 **REGRA DE SEGURANÇA Nº 0** — Clipping rígido a 15% da potência nominal. Leitura $\geq 20\%$ → Emergency Stop imediato e desligamento total.

- Transplante da eletrônica da bancada MDF para o chassi metálico do robô.
- Instalação dos optoacopladores (PC817) em todas as linhas PWM/DIR/BREAK.
- motor_driver.py ativação real: clipping validado em osciloscópio ANTES de energizar.
- Testes progressivos de potência: 5% → 10% → 15% máx, medindo corrente a cada degrau.
- Validação BNO085 + motores: linha reta de 2m com desvio máximo de 10cm.
- Validação LIDAR + movimento: obstáculo → parada confirmada em $\leq 40\text{ms}$.

Gate de Conclusão — critérios obrigatórios antes de avançar para a próxima fase:

- Emergency Stop $\geq 20\%$ testado e confirmado no osciloscópio (nunca passou de 15%)
- Linha reta de 2m com desvio $< 10\text{cm}$ em 5 tentativas consecutivas
- Parada por LIDAR em $< 40\text{ms}$ medida com câmera lenta ou osciloscópio
- Temperatura dos drivers ZS-X11H após 10min de operação dentro do limite
- Nenhuma interferência de ruído dos motores no barramento I2C durante movimento
- Regra 0 não violada em nenhuma linha de log durante sessão de 15min

Fase 4 — Navegação Autônoma SLAM (10–14 dias)

- slam_nav.py: Aurora SDK → mapa 2D + localização em tempo real + navegação point-to-point.
- Sistema de missões: POI destino → planejamento de rota → execução → retorno à base HOME.
- Testes de campo com alunos cadeirantes: teleoperação com feedback de vídeo ao vivo.
- Stress test de frota: 3 robôs autônomos simultâneos por 30min sem colisão ou deadlock.
- Escalabilidade: novo robô entra na frota apenas editando config.py ($< 15\text{min}$ de configuração).

Gate de Conclusão — critérios obrigatórios antes de avançar para a próxima fase:

- Robô navega rota Mesa → Cozinha → Base de forma 100% autônoma
- 3 robôs autônomos simultâneos sem colisão em 30min de operação
- Aluno cadeirante teleoperou com sucesso em teste real com público
- Novo robô adicionado à frota em < 15 minutos de configuração
- Regra 0 nunca violada em nenhum log de sessão de qualquer robô

3.1 Loop de Execução Crítico — 50Hz / 20ms

Etapa	Módulo	Ação
1. Percepção	safety_bumper + heading_lock + battery_monitor	Atualiza flags: bloqueio_frontal, yaw_error, tensão
2. Timeout joystick	joystick_reader.timed_out()	Sem pacote > 200ms → força velocidade = 0
3. REGRA 0	motor_driver._apply_safety_clip()	Potência ≥ 20% → Emergency Stop imediato
4. Correção de rumo	heading_lock.get_yaw_error()	Linha reta: micro-ajuste diferencial via Yaw
5. Filtro de colisão	state['blocked']	Avanço + bloqueio_frontal = True → para
6. Envio físico	motor_driver.set_speed()	PWM/DIR saturados em 15% → optoacop. → ZS-X11H

3.2 Como Usar o Claude Code no Cursor

Fluxo de trabalho: Cursor conectado à Pi via Remote SSH → Claude Code lê os arquivos diretamente → edita → você testa no terminal integrado → commit no GitHub somente ao concluir o Gate da fase.

- Passo 1: Abra o PROMPT_INICIAL.md e cole na primeira mensagem do Claude Code.
- Passo 2: Peça ao Claude Code para implementar o módulo da fase atual.
- Passo 3: Rode `python3 main.py --log DEBUG` no terminal integrado do Cursor.
- Passo 4: Itere com o Claude Code até o Gate da fase ficar 100% verde.
- Passo 5: `git commit -m 'feat: Fase N concluída' + git tag fase-N-concluída`.
- Passo 6: `git push origin main --tags` e inicie a próxima fase.